

Lab 4 - Vibe Coding: Chat History

Student: Girish G N (PES1UG24CS571) | Repo: 22_sliding_puzzle | AI tool used: Claude (claude.ai)

Note: this is a condensed transcript of the chat. Terminal outputs and screenshots pasted by the student are summarised in brackets. All prompts, explanations and code from the AI are included.

Part 1 - Understanding the assignment and setup

Girish: [Uploaded the Lab 4 handout PDF and the email screenshots]

https://github.com/SETAPESU26/22_sliding_puzzle - i got this puzzle, teach me step by step what to do: when to open what, what to run, when to take video. I don't know anything.

Claude: Fetched the repo README. It is a terminal Sliding Puzzle with 4 tasks: (1) guarantee solvable starting states, (2) complete puzzle lifecycle, (3) 3x3/4x4/5x5 modes with timer and moves, (4) valid-action feedback. Gave an overview of the lab workflow: install check, fork, clone, run, record before video, fix with prompts, one commit per task, record after video, push to personal repo, export chat.

Girish: [Asked to go one step at a time instead of all at once]

Claude: Went step by step. Checked Python (3.11.6) and Git (2.52.0) were installed.

Girish: [Forked SETAPESU26/22_sliding_puzzle to GIRISH-G-N/22_sliding_puzzle, cloned it to D:\SE\LAB-04, ran python main.py]

Claude: Confirmed the game runs: 4x4 board, W/A/S/D moves a tile into the blank, Q quits. Explained how to record the 'before' video with Windows key + Alt + R.

Girish: [Recorded the before video, saved as before.mp4]

Part 2 - Inspecting the code

Girish: [Pasted puzzle.py and game.py from the terminal using the type command]

Claude: Inspected the code and found the problems:

- puzzle.py: random.shuffle(tiles) creates a fully random board; only half of all arrangements are solvable (Task 1 bug, marked 'intentional solvability defect' in the code).
- game.py: size is fixed at 4, no solved-state handling beyond printing 'Solved!', timer keeps running, empty input could crash (key not in 'wasd' accepts an empty string).
- Task 3 and Task 4 features were missing or incomplete.

Part 3 - Task 1: Guarantee solvable starting states

Girish: [Asked the AI to write the code directly; the student will paste, run and screenshot]

Claude: Prompt/approach: start from the solved board and scramble it using only legal blank moves (never undoing the previous move, and re-scrambling if the result happens to be the solved board). Provided the new puzzle.py:

```
import random
```

```
class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def solved_board(self):
        tiles = list(range(1, self.size * self.size)) + [0]
        return [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]

    def make_board(self):
        # Start from the solved board and apply only legal blank moves,
        # so every generated board is guaranteed to be solvable.
        self.board = self.solved_board()
```

```

opposite = {"w": "s", "s": "w", "a": "d", "d": "a"}
while True:
    last = None
    steps = 0
    while steps < self.size * self.size * 20:
        direction = random.choice("wasd")
        if last and direction == opposite[last]:
            continue # don't immediately undo the previous move
        if self.move(direction):
            last = direction
            steps += 1
    if not self.solved():
        return self.board

def blank_pos(self):
    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c

def move(self, direction):
    r, c = self.blank_pos()
    dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}[direction]
    nr, nc = r + dr, c + dc
    if not (0 <= nr < self.size and 0 <= nc < self.size):
        return False
    self.board[r][c], self.board[nr][nc] = self.board[nr][nc], self.board[r][c]
    return True

def solved(self):
    return sum(self.board, []) == list(range(1, self.size * self.size)) + [0]

```

Girish: [First attempt: the file was not saved, so git commit said 'nothing added to commit'; findstr solved_board puzzle.py printed nothing]

Claude: Corrected my earlier claim that Task 1 worked - a random-looking board proves nothing if the old code is still running. Re-did the edit, verified with findstr that solved_board exists in puzzle.py.

Girish: [Verified findstr output, committed: 'Task 1: guarantee solvable starting states']

Part 4 - Task 2: Complete puzzle lifecycle

Claude: Updated game.py to detect the solved board, freeze the timer, print a final summary and stop accepting moves after completion:

```

import time
from puzzle import Puzzle

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.finished = False
        self.final_time = 0

    def elapsed(self):
        if self.finished:
            return self.final_time
        return int(time.monotonic() - self.started)

    def finish(self):
        self.finished = True
        self.final_time = int(time.monotonic() - self.started)

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print("Moves:", self.moves, " Time:", self.elapsed(), "s")

    def run(self):

```

```

print("Sliding Puzzle - W/A/S/D moves the tile into the blank. Q quits.")
while True:
    self.display()
    if self.finished:
        print(f"Solved in {self.moves} moves and {self.final_time} s. Game over!")
        return
    key = input("> ").strip().lower()
    if key == "q":
        return
    if key not in "wasd":
        print("Use W/A/S/D.")
        continue
    if self.puzzle.move(key):
        self.moves += 1
        if self.puzzle.solved():
            self.finish()
    else:
        print("That move is not possible.")

```

Girish: [Tested with a board one move from solved: `python -c "from game import SlidingPuzzle; ... board=[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,0,15]] ..."` then typed d]

Claude: Output showed the solved board and 'Solved in 1 moves and 2 s. Game over!' and the game exited. Task 2 passed. Committed: 'Task 2: detect solved state and end game cleanly'.

Part 5 - Task 3: Size, timer and move modes

Claude: Updated game.py: asks for a board size (3, 4, 5) at start; typing 3, 4 or 5 during play switches size by creating only a new board, so Moves and the timer carry over. Also fixed the empty-input crash by checking key in ('w','a','s','d').

Girish: [Tested: chose 3x3, made 4 moves, typed 5 -> board changed to 5x5 with 'Moves: 4' kept and timer continuing; typed x -> 'Use W/A/S/D, 3/4/5 or Q.']

Claude: Task 3 passed. Committed: 'Task 3: add 3x3, 4x4, 5x5 modes with persistent moves and timer'.

Part 6 - Task 4: Valid-action feedback

Claude: Two small edits. (1) In puzzle.py, move() now returns the tile that slid (return self.board[r][c]) instead of True, and still returns False for impossible moves. (2) In game.py, success is only reported and Moves only incremented when a tile actually moved. Final game.py:

```

import time
from puzzle import Puzzle

SIZES = ("3", "4", "5")

class SlidingPuzzle:
    def __init__(self, size=4):
        self.size = size
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.finished = False
        self.final_time = 0

    def new_board(self, size):
        # Only the board is recreated; moves and the timer are kept.
        self.size = size
        self.puzzle = Puzzle(size)

    def choose_size(self):
        while True:
            choice = input("Choose board size (3, 4 or 5): ").strip()
            if choice in SIZES:
                self.new_board(int(choice))
                return
            print("Please enter 3, 4 or 5.")

    def elapsed(self):

```

```

    if self.finished:
        return self.final_time
    return int(time.monotonic() - self.started)

def finish(self):
    self.finished = True
    self.final_time = int(time.monotonic() - self.started)

def display(self):
    print()
    for row in self.puzzle.board:
        print(" ".join(f"{x or ' ':>2}" for x in row))
    print("Moves:", self.moves, " Time:", self.elapsed(), "s")

def run(self):
    print("Sliding Puzzle - W/A/S/D moves the tile into the blank.")
    print("Type 3, 4 or 5 to change board size. Q quits.")
    self.choose_size()
    self.started = time.monotonic()
    while True:
        self.display()
        if self.finished:
            print(f"Solved in {self.moves} moves and {self.final_time} s. Game over!")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key in SIZES:
            self.new_board(int(key))
            print(f"Switched to {key}x{key}. Moves and time carried over.")
            continue
        if key not in ("w", "a", "s", "d"):
            print("Use W/A/S/D, 3/4/5 or Q.")
            continue
        tile = self.puzzle.move(key)
        if tile:
            self.moves += 1
            print(f"Slid tile {tile} into the blank.")
            if self.puzzle.solved():
                self.finish()
        else:
            print("That move is not possible. Move count unchanged.")

```

Girish: [Tested: 'w' printed 'Slid tile 3 into the blank.' and Moves went 0 -> 1; 'a' (impossible) printed 'That move is not possible. Move count unchanged.' and Moves stayed at 1]

Claude: Task 4 passed. Committed: 'Task 4: report success only when a tile actually moves'.

Part 7 - After video and submission

Girish: [Recorded the after video: 3x3 board, valid moves with 'Slid tile ...' messages, an invalid move with unchanged count, switch to 5x5 keeping Moves and Time. Pushed commits with git push to [GIRISH-G-N/22_sliding_puzzle.](#)]

Claude: Created the Lab-4 folder containing before.mp4, after.mp4, main.py, game.py, puzzle.py, requirements.txt, README.md and this chat history, then copied it into the SE-Labs-PES1UG24CS571 repo to push. No pull request was raised to SETAPESU26.

Summary of commits (one per task)

1. Task 1: guarantee solvable starting states
2. Task 2: detect solved state and end game cleanly
3. Task 3: add 3x3, 4x4, 5x5 modes with persistent moves and timer
4. Task 4: report success only when a tile actually moves